Linear Support Vector Machines

2026-04-17

Introduction

Linear support vector machines (SVMs) find the hyperplane that maximally separates classes with the largest possible margin. The soft-margin formulation introduced by Cortes and Vapnik in 1995 added slack variables to tolerate misclassified or margin-violating points, governed by a regularisation parameter C. Linear SVMs scale to very large sparse feature spaces (common in text classification, where bag-of-words gives millions of features) and are competitive with logistic regression on most tabular classification tasks.

Prerequisites

A working understanding of linear classification, convex optimisation, and the geometric concept of margin in supervised learning.

Theory

The soft-margin primal optimisation problem is

$$\min_{w,b,\xi} \tfrac{1}{2}\|w\|^2 + C\sum_i \xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The first term maximises the margin (minimising $\|w\|$); the second penalises slack. Larger C penalises slack more, favouring low training error at the cost of higher variance; smaller C allows more margin violations and produces a smoother boundary. The dual formulation exposes the support-vector structure: only points on or inside the margin contribute to w, so the trained model is effectively determined by a sparse subset of training points.

Assumptions

Classes are approximately linearly separable for the chosen C; features are appropriately scaled (SVMs are distance-based and unscaled features dominate the geometry).

R Implementation

```
library(e1071)

set.seed(2026)
n <- 200
x1 <- rnorm(n); x2 <- rnorm(n)
y <- factor(ifelse(x1 + x2 + rnorm(n, 0, 0.5) > 0, "pos", "neg"))
df <- data.frame(x1, x2, y)

# Scale the features
df[, 1:2] <- scale(df[, 1:2])

svm_fit <- svm(y ~ ., data = df,
```

```
                kernel = "linear",
                cost = 1,
                scale = FALSE)

table(pred = predict(svm_fit, df), truth = df$y)

# Vary C and observe support-vector count
cs <- c(0.01, 0.1, 1, 10, 100)
sapply(cs, function(c) {
  f <- svm(y ~ ., data = df, kernel = "linear", cost = c, scale = FALSE)
  c(C = c, n_sv = f$tot.nSV)
})
```

Output & Results

`svm()` returns a fitted classifier with the support vectors, dual coefficients, and a learned weight vector. Varying C shows the trade-off: small C retains many support vectors (soft boundary), large C retains few (tight boundary, high variance).

Interpretation

A reporting sentence: "A linear SVM with cross-validated $C = 1$ achieved 92 % test accuracy with 124 support vectors out of 200 training points; the support-vector count indicates the classes overlap moderately and a soft boundary is appropriate. Higher C values reduced support-vector counts but increased test error, confirming the bias-variance trade-off." Always report C, support-vector counts, and how C was tuned.

Practical Tips

- Standardise features before fitting; unscaled features with different units bias the geometry of the margin.
- Tune C by cross-validation over a logarithmic grid (typical range 10^{-3} to 10^{3}); the U-shaped CV-error curve has a clear optimum.
- For class imbalance, use `class.weights` to up-weight the minority class; alternatively, balance via under-sampling.
- For very large sparse feature spaces (text classification with millions of features), `LiblineaR` is dramatically faster than `e1071::svm` and equally accurate.
- Linear SVM and logistic regression often produce nearly identical decision boundaries on the same data; the SVM uses hinge loss while logistic uses log-loss, but for most tasks the choice is convention rather than substance.
- For non-linearly separable data, switch to kernel SVM (RBF, polynomial); the linear SVM cannot represent non-linear boundaries regardless of C.

R Packages Used

`e1071::svm()` for the canonical SVM implementation; `LiblineaR` for fast linear SVMs on large sparse data; `kernlab::ksvm()` for an alternative with richer kernel support; `parsnip::svm_linear()` for tidymodels integration; `caret` for cross-validated C tuning.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**

- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis